

Document Number: P2852R0
Date: 2023-04-24
Audience: SG21
Reply-to: Tom Honermann <tom@honermann.net>

Contract violation handling semantics for the contracts MVP

- [Introduction](#)
- [Proposal](#)
- [Design considerations](#)
 - [Predicate is unevaluated](#)
 - [Predicate evaluation yields a true result](#)
 - [Predicate evaluation yields a false result](#)
 - [Predicate evaluation encounters undefined-behavior](#)
 - [Predicate evaluation or violation handler throws](#)
 - [Predicate evaluation or violation handler calls `std::longjmp`](#)
- [Implementation experience](#)
- [Acknowledgements](#)
- [References](#)
- [Wording](#)

Introduction

SG21 endeavors to specify a minimal contracts design that is both simple enough to be confidently adopted for C++26 and sufficiently extensible to satisfy both known and unknown future needs.

Recent SG21 discussions have been focussed on the semantics to ascribe to behaviors that may occur during evaluation of contract predicates. For details of these discussions, see the minutes and papers linked below for recent SG21 telecons.

- [2023-03-09 SG21 telecon](#)
 - [P2698R0: Unconditional termination is a serious problem](#) ^[P2698R0]
 - [P2784R0: Not halting the program after detected contract violation](#) ^[P2784R0]
- [2023-03-23 SG21 telecon](#)
 - [P2817R0: The idea behind the contracts MVP](#) ^[P2817R0]
- [2023-04-20 SG21 telecon](#)
 - [P2838R0: Unconditional contract violation handling of any kind is a serious problem](#) ^[P2838R0]
 - [P2811R1: Contract-Violation Handlers](#) ^[P2811R1]

SG21 has not yet established consensus regarding:

1. which behaviors that occur during predicate evaluation are well-defined.
2. which behaviors that occur during execution of a violation handler are well-defined.

For the behaviors enumerated in [P2811R1](#)^[P2811R1], this paper proposes which ones to specify with well-defined semantics for the MVP, and which ones to be relegated to unspecified or undefined behavior pending further consideration.

It is not the intent of this paper to propose these semantics as the final set for C++26. The intent is rather to specify well-defined semantics for those cases for which strong consensus is established and to leave the rest to be pursued in future papers. In the meantime, the use of unspecified behavior leaves implementations free to experiment with semantics in the cases that are not well-defined.

This paper does not discuss how violation handlers are defined or registered, or the mechanics of their invocations; those concerns are covered by [P2811R1](#)^[P2811R1]. Likewise, this paper does not discuss build modes, the details of contract checking enablement, optimization based on contract predicates or consequences thereof, or one-definition rule (ODR) concerns regarding mixed contract checking enablement across the translation units that comprise a program; those concerns are also left to other papers.

Proposal

1. Adopt the contract violation handler specified in [P2811R1](#)^[P2811R1] with or without minor adjustments.
2. The implementation provided default contract violation handler is specified to unconditionally call `std::abort()` after performing any implementation-defined actions such as generating diagnostics.
3. A user-defined contract violation handler may call the implementation provided default contract violation handler. This ability enables a user-provided replacement handler to perform some actions and then defer to the implementation provided handler for implementation-specific generation of diagnostics. The mechanism by which this can be done is TBD since [P2811R1](#)^[P2811R1] does not expose the implementation provided handler when a user-provided handler is present.
4. The semantics of predicate evaluation and contract violation handler invocation are specified as follows.

Behavior	Specified semantics
Predicate evaluation is not performed.	well-defined ; execution continues.
Predicate evaluation yields a true result.	well-defined ; execution continues.
Predicate evaluation yields a false result.	well-defined ; the contract violation handler is invoked and, if it returns, execution continues.
Predicate evaluation encounters undefined-behavior	undefined behavior .
Predicate evaluation throws.	unspecified behavior .
Predicate evaluation calls <code>std::longjmp</code>	well-defined ; execution continues as specified by <code>std::longjmp</code> .
Violation handler terminates (calls <code>std::abort()</code> , <code>std::exit()</code> , <code>std::quick_exit()</code> , <code>std::terminate()</code> , etc...)	well-defined ; the program terminates as specified by the called function.
Violation handler returns	well-defined ; execution continues.
Violation handler throws	unspecified behavior .
Violation handler calls <code>std::longjmp</code>	well-defined ; execution continues as specified by <code>std::longjmp</code> .

Design considerations

The following discussion provides the rationale for each of the proposed semantics enumerated in the [proposal](#) section above.

The proposed design avoids the need for the C++ standard to acknowledge or to specify build modes. As always, implementations may offer options to optimize code generation based on an assumption that a contract violation handler will not return nor throw an exception.

Predicate is unevaluated

When predicate evaluation is not enabled, it seems clear that there is just one reasonable choice of semantics; that execution enters the function body. However, that does not mean that an implementation should completely ignore predicates; logic errors that involve predicates can still be diagnosed and can still affect observed program behavior. The following example contains such an error; the precondition unconditionally dereferences parameter `p` but the function body has behavior that is conditional on `p` being a null pointer. Implementors should be encouraged to diagnose such issues as a matter of QoI. Static analysis tools commonly detect and report issues like this.

```
int f(int *p)
    PRE( *p < 5 )
{
    if (!p)
        return 0;
    return *p;
}
```

Note that the logic error in the above example matches the one in the following example using `assert`. One of the benefits that contracts brings is that observance of the logic error is not dependent on a build mode; the predicate above is always present whereas the expression in the `assert` expression below is hidden from the compiler when compiling with `NDEBUG` defined.

```
int f(int *p)
{
    assert( *p < 5 );
    if (!p)
        return 0;
    return *p;
}
```

Predicate evaluation yields a true result

When predicate evaluation is enabled and evaluation yields a true result, it is likewise clear that the desired behavior is that execution enters the function body. The concerns about potential logic errors above are likewise applicable.

Predicate evaluation yields a false result

When predicate evaluation is enabled and evaluation yields a false result, this proposal calls for a contract violation handler to be invoked. Unlike [P2811R1](#)^[P2811R1], this proposal delegates all responsibility for handling the violation to the handler. If program termination is desired (the "enforce" or "Eval_and_abort" semantic described in other papers), then it is up to the handler to perform that termination (perhaps by calling the implementation-provided handler). If it is desired to continue program execution (the "observe" or "Eval_and_log" semantic), then the handler can simply return; this is consistent with the behavior of a contract violation when contract checking is not enabled.

Predicate evaluation encounters undefined-behavior

It is unlikely to be surprising that the proposed semantic for undefined-behavior is undefined-behavior. This case is explicitly called out to acknowledge previous discussion of [P2680R1](#)^[P2680R1]. Though consensus for that proposal has not been established, the potential for another proposal that seeks to constrain undefined-behavior within contract predicates to attain consensus is well recognized. In the meantime, this proposal reflects the status quo.

Predicate evaluation or violation handler throws

It remains unclear what semantics are desired in the case of exceptions that propagate from a predicate evaluation or violation handler invocation; particularly with regard to a function declared with a `noexcept` exception specification. Does such an exception originate from within the function such that it immediately hits the `noexcept` boundary? Or is it considered to have been thrown from the caller side of the function as is the case for exceptions thrown during parameter initialization? If the latter, how does that interact with the `noexcept` operator? Due to these open questions, this paper proposes that throwing an exception from a predicate or a violation handler have unspecified behavior until consensus is established for a direction as [P2780R0](#)^[P2780R0] seeks to do.

Predicate evaluation or violation handler calls `std::longjmp`

The open questions regarding exception handling don't apply to calls to `longjmp` since questions of `noexcept` exception specifications and the `noexcept` operator don't apply. The absence of a reason to deviate `longjmp` behavior in contract contexts therefore motivates making use in these contexts well-formed.

Implementation experience

None.

Acknowledgements

Thank you to Josua Berne for his excellent exposition of possible contract violation behaviors in [P2811R1](#)^[P2811R1].

References

- [P2680R1] Gabriel Dos Reis, "Contracts for C++: Prioritizing Safety", P2680R1, 2022.
<https://wg21.link/p2680r1>
- [P2698R0] Bjarne Stroustrup, "Unconditional termination is a serious problem", P2698R0, 2022.
<https://wg21.link/p2698r0>
- [P2780R0] Ville Voutilainen, "Caller-side precondition checking, and `Eval_and_throw`", P2780R0, 2023.
<https://wg21.link/p2780r0>
- [P2784R0] Andrzej Krzemiński, "Not halting the program after detected contract violation", P2784R0, 2023.
<https://wg21.link/p2784r0>
- [P2811R1] Joshua Berne, "Contract-Violation Handlers", P2811R1, 2023.
<https://wg21.link/p2811r1>

[P2817R0] Andrzej Krzemieński, "The idea behind the contracts MVP", P2817R0, 2023.

<https://wg21.link/p2817r0>

[P2838R0] Ville Voutilainen, "Unconditional contract violation handling of any kind is a serious problem", P2838R0, 2023.

<https://wg21.link/p2838r0>

Wording

Wording will be provided in a future revision contingent on SG21 establishing consensus for a proposed direction.